

TalentMatch

Plateforme intelligente de matching CV / Offres d'emploi

Rapport Technique

Choix technologiques, architecture et Implémentation

Réalisé par :	Youssef Gara
Entreprise :	3S Group — Tunis
Encadrante société :	Mme Sirine Nasri
Encadrante académique :	Mme Loujayne Bouzarti (Esprit)
Période :	Février 2026 — Août 2026
Date :	Juin 2026

1. Contexte et objectif du projet

1.1 Problématique

Le recrutement moderne souffre d'un paradoxe : les entreprises reçoivent un volume croissant de candidatures, mais les outils existants (LinkedIn, Indeed, ANAPEC) se limitent à une recherche par **mots-clés**, sans comprendre le sens réel des profils.

Chiffre clé

- Un recruteur consulte un CV en moyenne **23 secondes**.
- Sans outil intelligent, de nombreux bons profils sont ignorés ou mal classés.

Problèmes concrets identifiés :

- **Faux positifs** : un profil qui cite un mot-clé sans maîtriser le domaine.
- **Bons profils ignorés** : un expert qui formule ses compétences différemment.
- Pas de compréhension du **contexte sémantique** ni de la trajectoire professionnelle.
- **Absence d'explication** : le recruteur ne sait pas pourquoi un candidat est classé 1er.

1.2 Objectif — TalentMatch

TalentMatch est une plateforme web complète qui automatise le **matching CV / offres d'emploi** via un pipeline IA en 4 étapes :

- **Extraction** — lire un CV (PDF, DOCX, scan) et en extraire le texte brut.
- **Parsing NLP** — structurer le texte en JSON (compétences, expériences, formations).
- **Scoring IA** — calculer un score de pertinence 0–100% par modèle d'embeddings.
- **Classement et explication** — classer les candidats et afficher un rapport détaillé.

2. Architecture générale du système

2.1 Vue d'ensemble

Le système suit une architecture **client-serveur découplée**. Le frontend React communique uniquement avec l'API FastAPI. L'API orchestre les services IA, NLP et la base de données.

Couche	Technologie	Rôle
Frontend	React 18 + Vite	Interface utilisateur (SPA)
API	FastAPI (Python)	Logique métier, endpoints REST
IA / NLP	spaCy, BGE-M3, MLP	Parsing et matching
Base de données	PostgreSQL 16	Stockage persistant (port 5433)
Auth	JWT (python-jose)	Sécurité des accès
Email	SMTP Gmail	Notifications
Déploiement	Railway + Docker	Cloud

2.2 Fichiers principaux

Structure du projet – 3s-talentmatch/
<pre>backend/app/main.py <- Point d'entrée FastAPI (lifespan, CORS, routes) backend/app/routes/ <- Endpoints REST (auth, cv, matching, job_offers...) backend/app/models/ <- Tables SQLAlchemy (ORM) backend/app/schemas/ <- Validations Pydantic backend/app/services/ extraction/ <- Lecture PDF/DOCX/OCR nlp/ <- Pipeline NLP (spaCy) matching/match_engine.py <- Moteur de matching principal matching_sandbox/bert_scorer.py <- Scoreurs IA (BGE-M3 + MLP) ai_summary/claude_summarizer.py <- Résumé IA du profil backend/scripts/ <- Entraînement MLP, seed BDD frontend/src/ App.jsx <- Routing React services/api.js <- Instance Axios centralisée (JWT auto) pages/ <- Dashboard, OfferDetail, OffersList... components/MatchReport.jsx <- Rapport de matching affiché au recruteur</pre>

3. Choix technologiques et justifications

3.1 Backend — FastAPI

FastAPI est le framework Python choisi pour l'API REST. Il est basé sur **ASGI** (Starlette) et utilise les types Python natifs pour la validation.

Pourquoi FastAPI et pas Flask ou Django ?

- **Async natif (ASGI)** : gère plusieurs requêtes simultanées sans bloquer — crucial pour les appels aux modèles IA qui peuvent prendre 1–3 secondes.
- **Documentation automatique** : Swagger UI sur /docs et ReDoc sur /redoc, générés depuis les types Python. Pas de travail supplémentaire.
- **Validation automatique** : les schémas Pydantic valident les entrées/sorties et génèrent des messages d'erreur clairs.
- **Flask** : trop bas niveau, synchrone par défaut, pas de validation intégrée.
- **Django** : monolithique (ORM, admin, templates), overhead inutile pour une API pure.

Fichier clé

- backend/app/main.py — point d'entrée FastAPI.
- Le lifespan handler pré-chauffe le modèle BGE-M3 au démarrage pour éviter la latence au premier appel de matching.

3.2 Base de données — PostgreSQL

Pourquoi PostgreSQL et pas MySQL ou MongoDB ?

- **ACID** : transactions fiables — important pour les opérations de matching qui mettent à jour plusieurs tables simultanément.
- **JSONB natif** : le champ parsed_data (profil structuré du candidat) est stocké en JSONB, ce qui permet des requêtes sur les sous-champs.
- **MongoDB** : schéma trop flexible, difficile à maintenir avec des relations (candidat ↔ offre ↔ match ↔ user).
- **Port 5433** (non-standard) pour éviter les conflits avec d'autres instances locales.

Tables principales :

Table	Description
users	Comptes (candidat/recruteur/admin), auth JWT + OAuth
candidates	Profil structuré + parsed_data JSONB
cv_documents	Fichier CV uploadé
job_offers	Offres d'emploi (titre, description, compétences)
matches	Score de matching candidat↔offre (0–100%)

notifications	Alertes email (nouvelles candidatures, statuts)
access_logs	Journal RGPD — chaque consultation de données personnelles

Fichiers clés — Modèles

- backend/app/models/candidate.py — modèle Candidate (parsed_data JSONB)
- backend/app/models/match.py — table des scores de matching
- backend/app/models/access_log.py — journal RGPD
- backend/alembic/versions/ — 9 fichiers de migration

3.3 Frontend — React 18 + Vite

Pourquoi React et pas Vue ou Angular ?

- **Composants réutilisables** : le composant MatchReport est utilisé à la fois dans le dashboard recruteur et la page candidat.
- **Hooks** (useState, useEffect, useContext) : gestion d'état sans Redux.
- **Vite** : serveur de développement ultra-rapide (HMR instantané).
- **Angular** : trop verbose, TypeScript obligatoire, courbe d'apprentissage.

Point critique — Instance Axios centralisée

- frontend/src/services/api.js — TOUJOURS importer depuis ce fichier.
- Il injecte automatiquement le JWT dans chaque requête et redirige vers /login en cas de 401.
- Contourner ce fichier casserait l'authentification silencieusement.

4. Pipeline d'ingestion des CVs

Le pipeline complet se déclenche sur **POST /api/upload-cv** (backend/app/routes/cv.py). Il comporte 3 étapes.

4.1 Extraction du texte (backend/app/services/extraction/)

Type de fichier	Outil	Fichier
PDF textuel	PyMuPDF / pypdf	pdf_extractor.py
DOCX (Word)	python-docx	word_extractor.py
Image / PDF scan	EasyOCR	ocr_extractor.py

Pourquoi EasyOCR et pas Tesseract ?

- EasyOCR utilise du **Deep Learning** (CRAFT + CRNN) — meilleure précision sur les documents bruités, inclinés ou avec des polices non standard.
- Tesseract est basé sur de l'OCR classique, plus fragile sur les CVs scannés.
- EasyOCR supporte le français et l'arabe nativement.

4.2 Parsing NLP (backend/app/services/nlp/nlp_parser.py)

L'orchestrateur NLPParser appelle les extracteurs dans l'ordre suivant :

1. **ContactExtractor** — nom, email, téléphone, adresse.
2. **EntityExtractor** — entités nommées via spaCy NER (+ CamemBERT optionnel).
3. **SkillsExtractor** — compétences techniques et soft skills. Fichier : backend/app/services/nlp/skills_extractor.py
4. **FormationExtractor** — diplômes, établissements, années. Fichier : backend/app/services/nlp/formation_extractor.py
5. **ExperienceExtractor** — postes, entreprises, durées. Fichier : backend/app/services/nlp/experience_extractor.py
6. **Langues** — langues détectées automatiquement (_extract_langues).

Pourquoi spaCy fr_core_news_md et pas NLTK ?

- spaCy propose une pipeline NLP complète (tokenisation, POS, dépendances, NER) en un seul modèle.
- fr_core_news_md est entraîné sur du français journalistique — bien adapté aux CVs.
- NLTK est un toolkit académique sans modèle NER français de qualité.
- Important : le modèle n'est PAS installable via pip. Il faut : python -m spacy download fr_core_news_md

4.3 Stockage structuré (Candidate.parsed_data)

Le résultat du parsing est stocké dans le champ JSONB **parsed_data** de la table candidates. Sa structure est définie dans **backend/app/schemas/cv_data.py**.

Exemple de structure parsed_data (JSON)

```
{
  "identite": {"nom_complet": "Ahmed Benali", "email": "ahmed@..."},
  "competences": {
    "techniques": ["React", "Python", "FastAPI"],
    "soft": ["Leadership", "Communication"]
  },
  "experiences": [{"poste": "Dev React Senior", "duree_mois": 36, ...}],
  "formations": [{"diplome": "Master Info", "etablissement": "ESPRIT"}],
  "secteur_detecte": {"secteur": "informatique", "label": "Développeur"},
  "metadata": {"spacy_fallback": false, "version": "2.1"}
}
```

5. Moteur de Matching IA

Le score final (0–100%) est calculé par le scoreur principal `backend/app/services/matching_sandbox/bert_scorer.py` via une architecture à 3 niveaux.

Niveau	Composant	Rôle
1	BGE-M3 (bi-encoder)	Embeddings sémantiques des textes
2	Cross-Encoder (reranker)	Score sémantique précis sur la paire
3	MLP Fusion (7 features)	Score final combiné 0–1

5.1 BGE-M3 — Bi-encoder

Qu'est-ce que BGE-M3 ?

- BAAI/bge-m3 — modèle d'embeddings multilingue du Beijing Academy of Artificial Intelligence.
- 568 millions de paramètres, vecteurs de dimension 1024.
- Supporte 100+ langues — dont français, arabe et anglais simultanément.
- Basé sur XLM-RoBERTa fine-tuné pour la similarité sémantique.
- Il encode séparément le CV et l'offre, puis calcule la similarité cosinus.

Pourquoi BGE-M3 et pas CamemBERT ?

- **CamemBERT** est monolingue (français uniquement). Les CVs tunisiens contiennent souvent de l'anglais, du français et parfois de l'arabe.
- BGE-M3 est spécifiquement entraîné pour la **recherche sémantique** et la **similarité de documents**, pas juste pour le NER ou la classification.
- CamemBERT était utilisé dans les versions antérieures (Sprint 2-3). Le passage à BGE-M3 au Sprint 4 a amélioré significativement les scores.

Localisation dans le code

```
Fichier : backend/app/services/matching_sandbox/bert_scorer.py
Ligne 734 : _EMBEDDER_MODEL = 'BAAI/bge-m3'
Ligne 758 : class BERTMatchingScorer:
-> méthode _encode() : calcule les embeddings
-> méthode score() : retourne la similarité cosinus
```

5.2 Cross-Encoder — Reranker

Qu'est-ce qu'un Cross-Encoder ?

- BAAI/bge-reranker-v2-m3 — le compagnon canonique de BGE-M3.
- Contrairement au bi-encoder, il prend la paire (offre, CV) comme entrée UNIQUE.
- Il peut capturer des interactions fines entre les deux textes.

- Résultat : un score scalaire plus précis que la similarité cosinus seule.
- Limitation : plus lent (pas de cache). Utilisé en reranker après filtrage BGE-M3.

Localisation dans le code

Fichier : backend/app/services/matching_sandbox/bert_scorer.py
 Ligne 737 : `_RERANKER_MODEL = 'BAAI/bge-reranker-v2-m3'`
 Ligne 794 : `def _load_reranker(self): -> lazy loading au premier appel`
 CrossEncoder chargé avec `max_length=512`

5.3 MLP Fusion — Score final

Architecture du MLP

Architecture — bert_scorer.py ligne 742

- 7 features → 64 neurones → 32 neurones → 1 score
- Activations : ReLU sur couches cachées, Sigmoid en sortie (score entre 0 et 1).
- Poids entraînés : data/mlp_training_fusion/fusion_mlp.pt

Les 7 features d'entrée

#	Feature	Description
1	s_sémantique	Score du Cross-Encoder sur la paire (offre, CV)
2	skills_ratio	Ratio compétences communes / compétences requises
3	years_match	Correspondance années d'expérience (pénalisé si insuffisant)
4	education_match	Correspondance niveau d'études (1.0 si ≥ requis)
5	domain_match	Compatibilité domaine ISCO-08 (1.0 = même, 0.35 = différent)
6	skills_count_ratio	Nombre de skills candidat / nombre requis par l'offre
7	keyword_density	Densité de mots-clés de l'offre trouvés dans le CV

Pourquoi un MLP et pas une formule pondérée fixe ?

- Une formule fixe (ex: 40% sémantique + 30% skills + 30% expérience) est **rigide** et ne s'adapte pas aux nuances métier.
- Le MLP **apprend les poids optimaux** depuis des données d'entraînement réelles.
- Il peut apprendre des interactions non-linéaires : ex. un candidat sans diplôme requis mais avec 10 ans d'expérience est quand même pertinent.

Entraînement du MLP

Fichiers d'entraînement

- backend/scripts/generate_mlp_dataset_fusion.py
- -> Génère 1380+ paires (candidat, offre) avec labels intelligents.
- -> Labels : 1.0 = match parfait, 0.0 = hors domaine, valeurs intermédiaires calculées.

-
- backend/scripts/train_mlp_fusion.py
- -> Entraîne le MLP sur ce dataset.
- -> Sauvegarde : data/mlp_training_fusion/fusion_mlp.pt
-
- Le modèle entraîné est chargé au démarrage via le lifespan handler (backend/app/main.py).

5.4 Domaine professionnel — ISCO-08 / EscoService

Qu'est-ce que l'ISCO-08 ?

- International Standard Classification of Occupations 2008 — classification de l'OIT.
- Couvre 53 domaines professionnels universels.
- Utilisé pour détecter si le CV et l'offre sont dans le même domaine.

Avant de calculer le score, le système plafonne selon la compatibilité des domaines (fichier : **backend/app/services/matching_sandbox/esco_service.py**) :

- **Même domaine** (ex: Dev React ↔ Offre React) → pas de cap, score libre.
- **Domaines adjacents** (ex: Data Science ↔ Dev Backend) → cap à 65%.
- **Domaines différents** (ex: Médecin ↔ Offre Dev) → cap à 35%.

Fichiers ISCO-08

- backend/app/services/matching_sandbox/esco_service.py — service de détection
- data/esco/isco08_fr.json — 53 groupes, keywords, skills_fr
- Méthode : EscoService.get_score_cap(domain1, domain2) -> float | None

5.5 Pipeline de scoring complet

Étape	Opération	Résultat
1	BGE-M3 encode CV + Offre séparément	Vecteurs 1024-dim
2	Similarité cosinus entre les vecteurs	Score sémantique brut
3	Cross-Encoder (CV, Offre) en entrée unique	Score sémantique précis
4	Calcul des 6 autres features	Tableau 7 valeurs
5	MLP Fusion prédit le score final	Score 0–1
6	Plafonnement ISCO-08 si domaines différents	Score ajusté
7	Multiplication × 100	Score % affiché

6. Sécurité et conformité RGPD

6.1 Authentification JWT

Pourquoi JWT et pas sessions classiques ?

- **Stateless** : le serveur n'a pas besoin de stocker les sessions en BDD. Scalable horizontalement (plusieurs instances du serveur).
- Déployable sur Railway sans partage de session entre instances.
- Token signé (HMAC-SHA256) — impossible à falsifier sans la clé secrète.

Implémentation

- backend/app/services/auth_service.py — génération et vérification du token
- backend/app/dependencies.py — get_current_user, get_current_admin
- Token envoyé dans : Authorization: Bearer
- Expiration : configurable via ACCESS_TOKEN_EXPIRE_MINUTES dans backend/.env

6.2 Trois rôles d'accès

Rôle	Accès autorisés
Candidat	Upload CV, voir ses candidatures, liste des offres publiques
Recruteur	Gérer ses offres, voir les résultats de matching, changer les statuts
Admin	Tout + gestion utilisateurs + logs RGPD + statistiques

6.3 Conformité RGPD

- **Consentement explicite** : le candidat coche une case RGPD lors de la candidature (visible sur postuleCv.png).
- **Journal d'accès** : table access_logs (backend/app/models/access_log.py) — chaque consultation de données personnelles est enregistrée (qui, quand, quoi).
- **Droit à l'effacement** : endpoint dédié de suppression complète des données.
- **Conservation limitée** : suppression automatique après 2 ans d'inactivité.
- **Accès restreint** : données candidats accessibles uniquement aux recruteurs authentifiés.

7. Interface utilisateur (Frontend)

7.1 Pages principales

Page / Fichier	Description
frontend/src/pages/Dashboard.jsx	Tableau de bord recruteur
frontend/src/pages/OffersList.jsx	Liste des offres (côté candidat)
frontend/src/pages/OfferDetail.jsx	Détail d'une offre + bouton Postuler
frontend/src/pages/OfferNew.jsx	Création d'une nouvelle offre
frontend/src/components/MatchReport.jsx	Rapport de matching IA détaillé
frontend/src/components/MatchReport.css	Styles du rapport de matching

7.2 Rapport de matching — MatchReport

Le composant `frontend/src/components/MatchReport.jsx` affiche pour chaque candidat :

- Score global (0–100%) avec code couleur (vert $\geq 70\%$, orange 40–70%, rouge $< 40\%$).
- Décomposition par dimension : Compétences, Expérience, Formation, Sémantique.
- Compétences présentes vs compétences manquantes.
- Bouton « Analyse IA » → rapport narratif généré par LLM (Mistral AI).

8. Méthodologie — Agile Scrum

Le projet est organisé en **6 sprints d'1 mois** (Février – Août 2026). Reviews régulières avec Mme Sirine Nasri (encadrante société) et Mme Loujayne Bouzarti (encadrante académique).

Sprint	Mois	Statut	Livrables principaux
1	Fév 2026	✓ OK	CDC, conception UML, architecture, setup environnement
2	Mar 2026	✓ OK	Upload CV, extraction texte, OCR, stockage BDD
3	Avr 2026	✓ OK	Pipeline NLP spaCy, extraction skills/expériences
4	Mai 2026	En cours...	BGE-M3 + Cross-Encoder, matching, dashboard RH
5	Jun 2026	TODO	Sécurité renforcée, tests intégration, notifications
6	Jul–Août 2026	TODO	Déploiement Railway, documentation, soutenance

8.1 Outils de travail

- **Git / GitHub** — versioning du code.
- **VS Code** — développement frontend et scripts Python.
- **PyCharm** — développement backend FastAPI.
- **Postman** — test des endpoints API.
- **Trello** — gestion du backlog (user stories par sprint).
- **Railway** — hébergement cloud (déploiement prévu Sprint 6).

9. Avancement actuel — Sprint 4

9.1 Fonctionnalités livrées

- ✓ Authentification complète (JWT, OAuth Google/LinkedIn, vérification OTP email).
- ✓ Upload et extraction multi-format (PDF textuel, DOCX, PDF scanné via OCR).
- ✓ Pipeline NLP complet : compétences, expériences, formations, langues.
- ✓ Moteur de matching BGE-M3 + Cross-Encoder + MLP Fusion.
- ✓ Dashboard recruteur : liste des offres, classement des candidats par score.
- ✓ Interface candidat : liste des offres compatibles, candidature avec consentement RGPD.
- ✓ Score détaillé par dimension dans le composant MatchReport.
- ✓ Résumé IA du profil (Mistral/Claude via backend/app/services/ai_summary/).
- ✓ Notifications (structure implémentée).

9.2 Démo — Scénario de test

Comptes et données de démo

- Recruteur : selim@esprit.tn / Demo3S2026!
- Admin : admin1@esprit.tn / Demo3S2026!
- Offre démo : Developpeur React Senior
- 8 candidats avec scores de 93.7% (Ahmed Benali) à 5.3% (Amina Diallo)

10. Perspectives

- 1. **Notifications email recruteur** — alerte automatique à chaque nouvelle candidature.
- 2. **Planification d'entretiens** — créneaux proposés par le recruteur, acceptés par le candidat.
- 3. **Page candidat enrichie** — compétences manquantes identifiées, conseils personnalisés.
- 4. **Amélioration continue par feedback** — notes recruteur → réentraînement du MLP.
- 5. **Génération de rapports via LLM (Mistral AI)** — rapport narratif complet par candidat.
- 6. **Déploiement Railway** — conteneurisation Docker, CI/CD GitHub Actions.

11. Lancement du projet — Commandes essentielles

Cette section liste les commandes exactes pour démarrer les serveurs backend et frontend, ainsi que les vérifications à faire si quelque chose ne fonctionne pas.

11.1 Prérequis — Vérifications avant de lancer

Vérifier que PostgreSQL tourne sur le port 5433

```
# Windows PowerShell :  
netstat -ano | findstr :5433  
  
# Si rien n'apparaît -> démarrer PostgreSQL :  
# Panneau de configuration > Services > postgresql-x64-16 > Démarrer
```

Vérifier que le venv Python est activé (backend)

```
# Depuis la racine du projet :  
.venv-10\Scripts\activate  
  
# Vérifier que FastAPI est installé :  
python -m pip show fastapi
```

11.2 Lancer uniquement le Backend

Répertoire : backend/ | Port : 8000

- URL API : <http://localhost:8000>
- Documentation : <http://localhost:8000/docs> (Swagger UI)
- Health check : <http://localhost:8000/health>

Commandes – Backend (depuis le dossier backend/)

```
cd backend  
  
# Appliquer les migrations si nécessaire (1ère fois ou après un changement de modèle) :  
alembic upgrade head  
  
# Lancer le serveur de développement :  
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000  
  
# Le flag --reload redémarre automatiquement à chaque modification du code.  
# Attendre le message : 'Application startup complete.'
```

11.3 Lancer uniquement le Frontend

Répertoire : frontend/ | Port : 3000

- URL interface : <http://localhost:3000>

- Le frontend proxy automatiquement /api/* vers http://localhost:8000
- Le backend DOIT être lancé pour que le frontend fonctionne.

Commandes – Frontend (depuis le dossier frontend/)

```
cd frontend

# Installer les dépendances (1ère fois uniquement) :
npm install

# Lancer le serveur de développement Vite :
npm run dev

# Le navigateur s'ouvre automatiquement sur http://localhost:3000
# Vite affiche 'VITE ready in XXXms' quand c'est prêt.
```

11.4 Lancer Backend ET Frontend ensemble (recommandé)

Script automatique — Windows uniquement

- Le script dev.ps1 lance les deux serveurs en arrière-plan en une seule commande.
- Les logs sont écrits dans logs/backend.log et logs/frontend.log
- Les PIDs sont sauvegardés dans logs/.dev-pids.json

Depuis la racine du projet (3s-talentmatch/)

```
# Lancer backend + frontend en parallèle :
.\dev.ps1

# Vérifier que tout tourne :
# - Backend : http://localhost:8000/health
# - Frontend : http://localhost:3000

# Arrêter les deux serveurs :
.\stop-dev.ps1
```

11.5 Variables d'environnement (.env)

Fichier : backend/.env – variables importantes

```
DATABASE_URL=postgresql://postgres:password@localhost:5433/talentmatch
SECRET_KEY=
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Email (notifications) :
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=
SMTP_PASSWORD=

# CamemBERT NLP optionnel (désactivé par défaut) :
HF_ENABLE_CAMEMBERT_NAME=0
```

11.6 Résolution des problèmes courants

Problème	Cause probable	Solution
Port 8000 déjà utilisé	Un autre processus tourne	taskkill /F /IM python.exe ou changer le port
Port 5433 refusé	PostgreSQL non démarré	Démarrer le service PostgreSQL (Services Windows)
ModuleNotFoundError	Venv non activé	.venv-10\Scripts\activate puis relancer
npm : command not found	Node.js non installé	Installer Node.js 18+ depuis nodejs.org
spaCy model missing	fr_core_news_md absent	python -m spacy download fr_core_news_md
401 Unauthorized sur /api/*	Token JWT expiré ou absent	Se reconnecter sur /login pour obtenir un nouveau token
Score matching = 0%	Modèle BGE-M3 non chargé	Vérifier les logs au démarrage (lifespan handler dans main.py)

Conclusion

TalentMatch apporte une réponse technique complète à la problématique du matching CV / offres d'emploi. Le pipeline IA (BGE-M3 + Cross-Encoder + MLP Fusion) dépasse les approches par mots-clés des solutions existantes en comprenant la sémantique des profils et des offres.

Les choix technologiques (FastAPI, PostgreSQL, React, spaCy) ont été guidés par des critères clairs : performance, maintenabilité et adéquation avec le périmètre du projet. L'architecture modulaire (pipeline-shaped) facilite les évolutions futures.

Le Sprint 4 marque le cœur du projet avec la livraison du moteur de matching IA. Les sprints 5 et 6 compléteront la solution avec la sécurité renforcée, les tests d'intégration et le déploiement en production sur Railway.

Youssef Gara — Stage PFE 3S TalentMatch — Juin 2026